O-Voxel: Native 3D Latents for Arbitrary Topology and PBR Materials

Anonymous CVPR submission

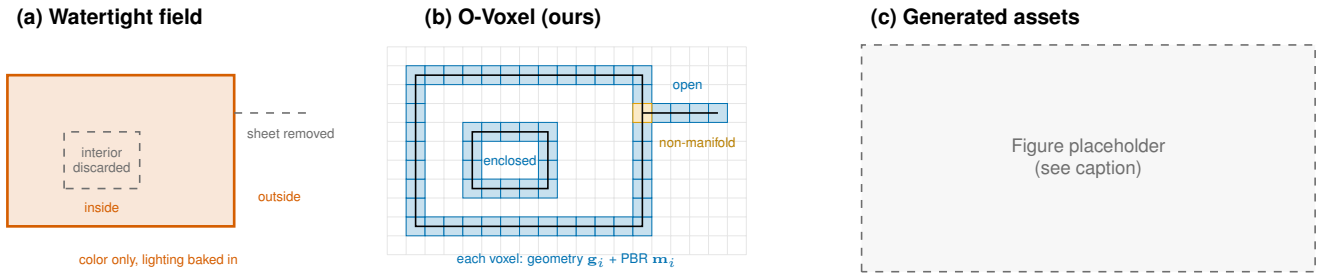


Figure 1. **Native 3D latents keep every surface and its material.** (a) An implicit field divides space into inside (orange) and outside, so the open sheet and the enclosed interior of an asset are lost when it is made watertight, and appearance is reduced to a lit color. (b) O-Voxel stores only the voxels that the surface passes through (blue), each with its geometry and PBR attributes; an open sheet, a non-manifold junction (yellow) and an enclosed surface are all sets of active voxels. (c) **[TODO: Figure plan: showcase of 6 to 8 generated assets with thin open parts, interiors and varied materials; for two of them show base color, metallic and roughness channels and a relit rendering; one row of inputs if generation is conditioned.]**

Abstract

Generative models of 3D assets have become markedly more realistic, yet what they can generate is still bounded by the representations they generate in. Implicit fields require a closed inside and outside, and appearance is usually reduced to a texture color, so open surfaces, non-manifold parts, fully enclosed interiors and physically based materials are lost before training begins. We instead learn a structured latent representation directly from native 3D data. At its core is O-Voxel, a sparse omni-voxel structure that stores geometry and appearance in the same voxels. O-Voxel represents arbitrary topology, including open, non-manifold and fully enclosed surfaces, and records physically based rendering parameters beyond texture color. A Sparse Compression VAE turns O-Voxels into a compact latent space with a high spatial compression rate, on which we train flow-matching models with 4B parameters on diverse public 3D asset datasets. Inference remains efficient at this scale, and the geometry and materials of the generated assets far exceed those of existing 3D generative models [TODO: benchmark and headline number].

1. Introduction

A usable 3D asset is more than a shape. Consider a car: its body is painted metal, its tires are rubber, its windows are thin sheets of glass, and its seats sit inside the closed shell of the cabin. Generative models of 3D assets now produce shapes and textures realistic enough for [TODO: target applications, e.g., games, film, simulation] [TODO: cite recent 3D generative models], but such an asset is still out of their reach as it is: thin open sheets, parts that meet along a shared edge, surfaces hidden inside other surfaces, and materials that respond correctly to light.

Much of the recent progress comes from compact latent representations on which diffusion and flow-matching models can be trained at scale [TODO: cite 3D latent generative models]. The main reason these models cannot produce such assets is the representation they generate in. Implicit fields describe a shape by its inside and outside [TODO: cite field-based 3D generation], so every training asset is first converted into a closed, watertight surface, a step that removes open sheets, merges non-manifold parts and discards interiors that no outside view can reach. Appearance is commonly stored as a single texture color [TODO: cite], which bakes lighting into the asset and leaves no place for the physically based rendering (PBR) parameters that production pipelines expect. Meshes with materials, the na-

tive format of 3D assets, keep all of this, but their irregular connectivity offers no fixed structure for a network to generate. Learnability and fidelity therefore pull in opposite directions.

Sparse voxels need not choose between them. Whatever its topology, the surface of an asset occupies a sparse set of cells in a regular grid, and each cell can describe the piece of surface that passes through it without any notion of inside or outside. We build on this observation with O-Voxel, an omni-voxel representation that stores geometry and appearance in the same sparse voxels (Fig. 1). An open sheet, a non-manifold junction and a surface enclosed by another are all sets of active voxels, and each voxel carries PBR parameters in addition to color. Because O-Voxel is computed directly from native 3D data [TODO: conversion from mesh and materials], no asset has to be closed, simplified or rendered before training.

The regularity of O-Voxel has a price: at the resolution that fine geometry needs, an asset covers [TODO: typical number of active voxels] voxels, too many for a generative model to process directly. We therefore design a Sparse Compression VAE (SC-VAE) that compresses O-Voxels spatially into a compact, structured latent space [TODO: compression mechanism and rate]. This compression is what makes scale affordable. We train flow-matching models with 4B parameters on diverse public 3D asset datasets [TODO: datasets], and their inference remains efficient because every sampling step operates on the compact latents.

Unlike 3D latent representations that are fitted to watertight fields or aggregated from rendered views [TODO: cite closest work], our latents are learned from native 3D data, so the generator sees the same open, non-manifold and enclosed surfaces and the same materials that it is asked to produce. The closest work is [TODO: name it], which differs from ours in [TODO: one checkable difference: supported topology, material channels or compression rate].

Our models generate assets whose geometry and materials far exceed those of existing 3D generative models [TODO: benchmarks, baselines and one or two headline numbers], while sampling an asset in [TODO: time] on [TODO: hardware]. O-Voxel with SC-VAE also reconstructs assets with open, non-manifold and enclosed surfaces that field-based representations cannot express (Sec. 3.2). We argue that the representation, not the generator, has been the bottleneck of 3D asset generation, and that a structured latent learned from native 3D data removes it.

2. Structured Latents from Native 3D Data

Given a 3D asset in its native form, a mesh with PBR materials [TODO: supported input formats], our goal is to learn a latent space in which a generative model can produce new assets of the same form. The approach has three parts (Fig. 2). O-Voxel turns an asset into a sparse set of voxels that carry both geometry and appearance (Sec. 2.1). The Sparse Compression VAE (SC-VAE) maps an O-Voxel to a compact latent and back (Sec. 2.2). Flow-matching models generate in this latent space, and the decoded O-Voxel is converted into a mesh with materials (Sec. 2.3).

2.1. O-Voxel: geometry and appearance in sparse voxels

A representation for generation must hold any surface that occurs in real assets and must have a regular structure that a network can produce. Fields satisfy the second requirement but not the first, because they classify space into inside and outside; meshes satisfy the first but not the second. O-Voxel keeps the regular grid of fields but drops the inside and outside: only the voxels that the surface passes through are stored.

Let the asset be normalized to a cube divided into a grid of resolution N^3 [TODO: value of N]. An O-Voxel is the set of active voxels

$$\mathcal{V} = \{(\mathbf{p}_i, \mathbf{g}_i, \mathbf{m}_i)\}_{i=1}^{L}, \tag{1}$$

where $\mathbf{p}_i \in \{0, \dots, N-1\}^3$ is the position of voxel i, $\mathbf{g}_i$ describes the surface inside the voxel [TODO: geometric attributes stored per voxel and how they locate the surface], and $\mathbf{m}_i$ holds its appearance [TODO: PBR attributes stored per voxel, e.g., which channels beyond base color]. A voxel is active when [TODO: activation rule]. The number of active voxels L grows with the area of the surface rather than with the volume of the grid, which is what makes high resolutions affordable.

Because no voxel needs an inside or outside label, O-Voxel imposes no condition on topology. An open sheet, such as a leaf or a window pane, is a single layer of active voxels, and a non-manifold edge, where three or more sheets meet, is a set of voxels that those sheets share. A surface fully enclosed by another, such as the interior of a cabin, is stored as its own active voxels even though no outside view reaches it (Fig. 1b). Appearance lives in the same voxels as geometry, so materials stay attached to the surface they belong to and are never baked into a lit texture. We construct $\mathcal{V}$ directly from the mesh and its material maps [TODO: construction procedure], and convert an O-Voxel back into a mesh with materials by [TODO: extraction procedure].

2.2. Sparse Compression VAE

At the resolution that fine geometry needs, L reaches [TODO: typical number of active voxels], which is too many tokens for a large generative model. SC-VAE compresses an O-Voxel spatially while keeping its sparse, structured form. Its encoder E maps $\mathcal{V}$ to a latent

$$\mathcal{Z} = E(\mathcal{V}) = \{(\mathbf{p}'_j, \mathbf{z}_j)\}_{j=1}^{L'}, \qquad L' \ll L, \tag{2}$$

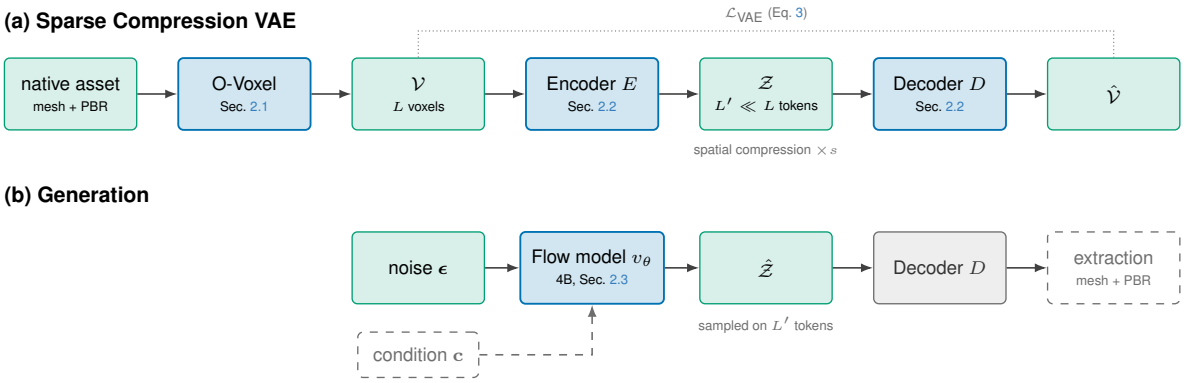


Figure 2. **Overview.** (a) A native asset is converted into an O-Voxel $\mathcal{V}$ of L active voxels; the SC-VAE encoder E compresses it into a structured latent $\mathcal{Z}$ of $L' \ll L$ tokens, and the decoder D restores the full-resolution O-Voxel. (b) A flow-matching model v_θ with 4B parameters transports noise to a latent $\hat{\mathcal{Z}}$, which D decodes into an asset with materials. Blue parts are ours, grey is reused, green is data; dashed parts are not yet specified (conditioning and extraction).

where $\mathbf{p}'_j$ are positions on a coarser grid of resolution $(N/s)^3$ with spatial compression factor s [TODO: value of s and how positions are pooled], and $\mathbf{z}_j \in \mathbb{R}^C$ are latent features [TODO: channel count C]. The decoder D restores the full-resolution O-Voxel, predicting which fine voxels are active together with their geometry and appearance [TODO: decoder architecture and upsampling scheme].

We train SC-VAE with a reconstruction objective on geometry and appearance and a regularizer on the latent,

$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{geo}} + \lambda_{\text{mat}}\,\mathcal{L}_{\text{mat}} + \lambda_{\text{KL}}\,\mathcal{L}_{\text{KL}}, \tag{3}$$

where [TODO: definitions of $\mathcal{L}_{\text{geo}}$, $\mathcal{L}_{\text{mat}}$ and any additional terms]. The compression factor s sets the tradeoff that the rest of the method depends on: a larger s gives the generator fewer tokens, while the decoder must recover more detail from each latent (Sec. 3.5).

2.3. Flow-matching generation

Because $\mathcal{Z}$ is compact, we can train a large generator on it. We use flow matching [TODO: cite], which learns a velocity field v_θ that transports noise to latents. For a latent $\mathbf{z}_0 = \mathcal{Z}$ from the training set, noise $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$ and time $t \in [0, 1]$, we form $\mathbf{z}_t = (1-t)\,\mathbf{z}_0 + t\,\boldsymbol{\epsilon}$ and minimize

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{t,\mathbf{z}_0,\boldsymbol{\epsilon}} \left\| v_\theta(\mathbf{z}_t, t, \mathbf{c}) - (\boldsymbol{\epsilon} - \mathbf{z}_0) \right\|_2^2, \tag{4}$$

where $\mathbf{c}$ is the condition [TODO: conditioning input: image, text or none]. [TODO: Confirm the interpolation path and time sampling; state whether sparse positions $\mathbf{p}'$ and features $\mathbf{z}$ are generated in one model or in separate stages.] The generators have 4B parameters [TODO: architecture, number of models and their roles] and are trained on diverse public 3D asset datasets (Sec. 3.1). At inference, we integrate v_θ from $t = 1$ to $t = 0$ in [TODO: number of steps] steps, decode the result with D and extract the asset. Every step operates on L' latent tokens instead of L voxels, so the cost of sampling is set by the compressed size of the asset.

Relation to field-based latents. SC-VAE and the generator follow the standard latent generative recipe; what changes is the space they operate in. A field-based latent can only decode what a watertight field can express, so its generator never sees open sheets, non-manifold parts, enclosed interiors or PBR channels. Our latents are learned from O-Voxels computed on native assets, so these properties are present in the training data of the generator and in every sample it decodes.

3. Experiments

Our experiments answer four questions. Can O-Voxel with SC-VAE reconstruct assets of arbitrary topology and their materials at a high compression rate (Sec. 3.2)? Do models trained in this latent space generate better geometry and materials than existing 3D generative models (Sec. 3.3)? Does inference stay efficient at 4B parameters (Sec. 3.4)? Which design choices are responsible (Sec. 3.5)?

3.1. Setup

Datasets. We train on diverse public 3D asset datasets [TODO: names, number of assets, filtering]. For evaluation we use [TODO: held-out sets and test benchmarks], and we additionally collect a topology test set with open, non-manifold and fully enclosed surfaces [TODO: confirm and describe] so that each representation is tested on the cases it is meant to cover.

Metrics. Geometry is measured with [TODO: e.g., Chamfer distance ↓, F-score ↑, normal consistency ↑]. Ma-

Table 1. **O-Voxel with SC-VAE reconstructs both geometry and materials at a high compression rate.** Reconstruction on [TODO: benchmark]; tokens are the latent length per asset. Best in **bold**.

Method	Tokens ↓	CD ↓	F1 ↑	PSNR ↑	LPIPS ↓
Baseline 1 ?	?	?	?	?	?
Baseline 2 ?	?	?	?	?	?
Baseline 3 ?	?	?	?	?	?
Ours	?	?	?	?	?

Table 2. **Our generated assets surpass existing models in geometry and materials.** Generation on [TODO: benchmark and conditioning]. Time is per asset on [TODO: hardware]. Best in **bold**.

Method	Params	Geometry ↑	Material ↑	Time (s) ↓
Baseline 1 ?	?	?	?	?
Baseline 2 ?	?	?	?	?
Baseline 3 ?	?	?	?	?
Ours	4B	?	?	?

terials are measured by rendering the PBR attributes under [TODO: lighting setup] and computing [TODO: e.g., PSNR ↑ and LPIPS ↓ per channel]. Generation quality is measured with [TODO: e.g., distribution metrics on renderings and a user study].

Baselines. For reconstruction we compare with the latent representations of [TODO: baseline representations]; for generation with [TODO: baseline 3D generative models]. All baselines are run with their official code and checkpoints on the same evaluation assets [TODO: confirm].

Implementation. [TODO: Grid resolution, compression factor, latent channels, optimizer, batch size, training steps and hardware for SC-VAE and for the generators.]

3.2. Reconstruction

O-Voxel with SC-VAE [TODO: state the finding: e.g., reconstructs geometry and materials more accurately than all baselines while using fewer tokens] (Tab. 1). The largest gap appears on [TODO: the subset or metric with the largest margin], because field-based representations must close every surface before encoding it, while O-Voxel encodes the surface as it is. Fig. 3 shows this on the topology test set: [TODO: describe what the baselines lose on open sheets, non-manifold edges and enclosed interiors].

3.3. Generation

Our models generate assets whose geometry and materials far exceed those of existing 3D generative models (Tab. 2),

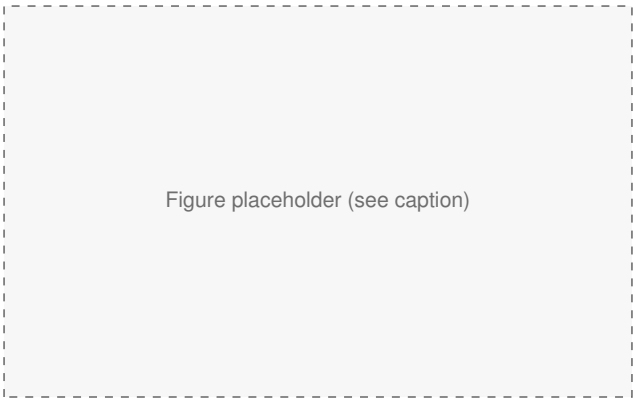


Figure 3. **Fields lose what O-Voxel keeps.** Reconstructions of assets with an open sheet, a non-manifold edge and an enclosed interior. **[TODO: Figure plan: 3 rows (one per topology case) × columns: ground truth, [TODO: 2 baseline representations], O-Voxel + SC-VAE; a cut-away view for the enclosed case and a zoom-in inset on the thin part; per-image Chamfer distance in the corner.]**

with [TODO: headline margin on the main geometry and material metrics]. The gain in material quality comes from the PBR attributes that O-Voxel stores per voxel: baselines that generate a texture color [TODO: describe what they miss, e.g., metallic or rough surfaces under new lighting]. Fig. 4 compares generated assets for the same inputs [TODO: describe the visible differences in thin structures, interiors and materials]. [TODO: If a user study is run: preference rates for geometry and materials.]

3.4. Efficiency

Despite 4B parameters, our models sample an asset in [TODO: time] (Tab. 2), [TODO: comparison with the fastest and the largest baseline]. The cost is low because generation runs on L' latent tokens: at compression factor $s =$ [TODO: value], an asset with [TODO: number] active voxels becomes [TODO: number] tokens. [TODO: Optional: time and memory as a function of grid resolution, with and without SC-VAE.]

3.5. Ablation study

Each ablation in Tab. 3 answers one question about the method. *Is O-Voxel responsible for the geometry gains?* Replacing it with a watertight field [TODO: effect], mostly on assets with open or enclosed surfaces (Sec. 2.1). *Do the PBR attributes matter?* Storing color only [TODO: effect on material metrics] (Sec. 2.1). *How much can SC-VAE compress?* Varying s [TODO: tradeoff between tokens and reconstruction] (Sec. 2.2). *Does scale help?* A smaller generator [TODO: effect] (Sec. 2.3).

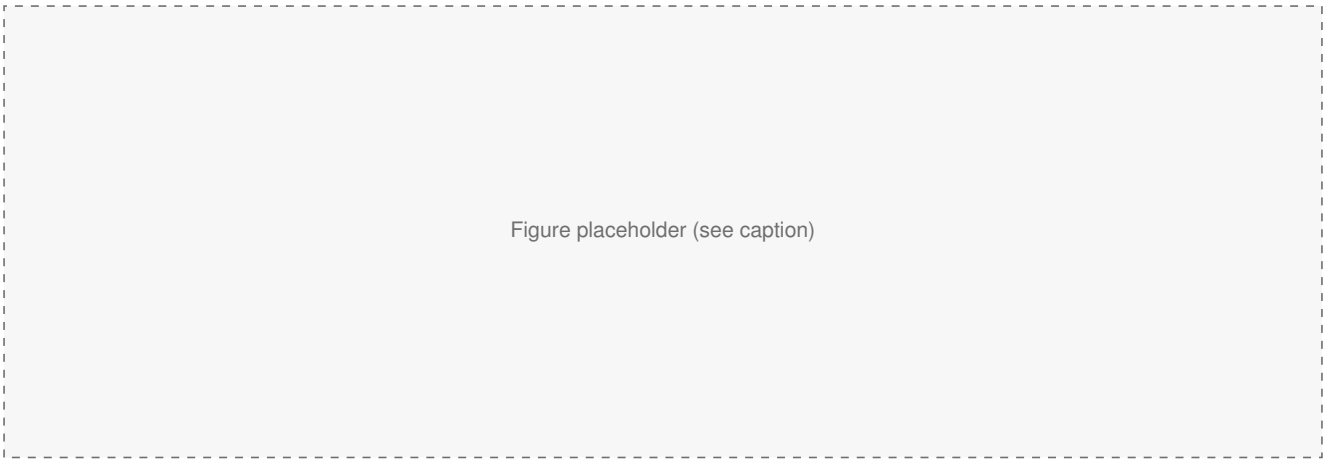


Figure 4. **Our assets keep thin parts, interiors and materials that baselines lose.** Generated assets for the same inputs, rendered with PBR materials under the same lighting. **[TODO: Figure plan: 4 rows (inputs chosen to stress thin open parts, interiors and metallic or rough materials) × columns: input [TODO: if conditioned], [TODO: 3 baselines], ours; same viewpoint and lighting for every method; one inset per row on the most telling region.]**

Table 3. **Each design choice contributes.** Ablations on [TODO: benchmark]; each row changes one component of the full model; ? marks values and baseline names still to fill.

Variant	CD ↓	Material ↑	Tokens ↓
Full model	?	?	?
Field instead of O-Voxel	?	?	?
Color only, no PBR	?	?	?
Smaller s ?	?	?	?
Larger s ?	?	?	?
Smaller generator ?	?	?	?

3.6. Limitations

[TODO: Concrete failure modes with a cause and, where possible, a figure. Candidates to verify are listed in the report: very thin features below the voxel size, the cost of the 4B models in training, transparent or layered materials, and license or bias properties of the public training data.]